_____

(наименование организации – заполнить)

**УТВЕРЖДАЮ**
Должность
_____
_____ / _____
«____» _____ 2026 г.
*(поля подписи оставлены пустыми)*

# ДЕТЕКТИВНАЯ ДОСКА (Detective Board)

Локальный офлайн нейро-символический следователь по рассказам о Шерлоке Холмсе

# ОПИСАНИЕ ПРОГРАММЫ

программного комплекса
ГОСТ 19.402-78

Обозначение документа: _____ (присваивается организацией)

Комплект программной документации ЕСПД, документ 3 из 8
Версия комплекса: 0.0.1 (pyproject.toml); состояние репозитория: main, коммит 16122fc
Дата составления: 30.10.2026. Статус: черновик, не согласован

2026

## ЛИСТ СОГЛАСОВАНИЯ

Подписи, фамилии и даты не заполнены: документ составлен по состоянию репозитория и не согласован ни с кем.

| Роль | Должность | Фамилия И. О. | Подпись | Дата |
|---|---|---|---|---|
| Разработал | | | | |
| Проверил | | | | |
| Нормоконтроль | | | | |
| Утвердил | | | | |

**СОДЕРЖАНИЕ**

1 ОБЩИЕ СВЕДЕНИЯ....4
2 ФУНКЦИОНАЛЬНОЕ НАЗНАЧЕНИЕ....4
3 ОПИСАНИЕ ЛОГИЧЕСКОЙ СТРУКТУРЫ....5
3.1 Архитектура: контекст и контейнеры....5
3.2 Модули программы....7
3.3 Поток подготовки данных....8
3.4 Поток обработки вопроса....9
3.5 Модель данных....10
3.6 Символический слой: правила R1–R3 и метки....12
3.7 MCP-контракт....13
3.8 HTTP-интерфейс....15
3.9 Веб-интерфейс....15
4 ИСПОЛЬЗУЕМЫЕ ТЕХНИЧЕСКИЕ СРЕДСТВА....16
5 ВЫЗОВ И ЗАГРУЗКА....16
6 ВХОДНЫЕ ДАННЫЕ....17
7 ВЫХОДНЫЕ ДАННЫЕ....17
ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ....18

Документ составлен по материалам репозитория и измерениям, лежащим в `docs/spike/results/` и `docs/perf/`, с помощью ИИ-ассистента; человеком не проверен и никем не согласован. Подписи, должности, даты согласования и обозначение документа оставлены пустыми намеренно.

**Условные обозначения статуса сведений в комплекте. [Измерено]** – число взято из файла отчёта, путь указан рядом. **[Не измерено]** – измерения нет. **[План]** – описано в проектных документах, в коде отсутствует или выключено. **[Заполнить]** – поле оставлено пустым для человека.

## 1 ОБЩИЕ СВЕДЕНИЯ

Таблица 1 – Сведения о программе

| Параметр | Значение |
|---|---|
| Обозначение | Detective Board, версия 0.0.1 (`pyproject.toml`); репозиторий `AndrewStrizhevskiy/detective-board` |
| Языки реализации | Python 3.12 (ядро, сервер, правила, конвейер), JavaScript ES-модули (интерфейс `web/`), shell (скрипты `scripts/`) |
| Необходимое программное обеспечение | uv, just, Ollama с моделями bge-m3, qwen3:4b-instruct, qwen3:4b; SQLite с FTS5; современный браузер |
| Библиотеки (`pyproject.toml`) | pydantic >=2.9,<3; sqlite-vec 0.1.9; mcp >=2.3,<3; starlette 1.9.0; uvicorn 0.54.0 |
| Состав | 8 подпакетов `src/detective_board/` (agent, api, extract, graph, index, mcp, retrieval, symbolic); тесты: 67 файлов `test_*.py` в `tests/` и тесты интерфейса в `tests/web_js/` |

## 2 ФУНКЦИОНАЛЬНОЕ НАЗНАЧЕНИЕ

Программа отвечает на вопросы по корпусу из 20 рассказов о Шерлоке Холмсе дословными цитатами, находит противоречия и проверяет алиби детерминированными правилами R1–R3, отказывается отвечать при отсутствии опоры в тексте и показывает ход расследования (граф фактов, трассу шагов, объяснения выводов). Классы решаемых задач: поиск по тексту (факт, цитата, кто сказал); вопросы о связях сущностей; хронология; алиби и таблица присутствия; противоречия; доска исключения по алиби для сцены преступления.

# 3 ОПИСАНИЕ ЛОГИЧЕСКОЙ СТРУКТУРЫ

## 3.1 Архитектура: контекст и контейнеры

Исходные описания – `docs/architecture/c4.md` (диаграммы Mermaid, раздел «Статус: черновик, фаза 0»). Диаграммы перерисованы в Graphviz и приведены к реализованному состоянию: граф хранится в SQLite (Ladybug не используется, ADR-0002), фреймворк интерфейса – без сборки, API – Starlette, символический слой реализован. Исходники диаграмм – `docs/gost/diagrams/*.dot`.

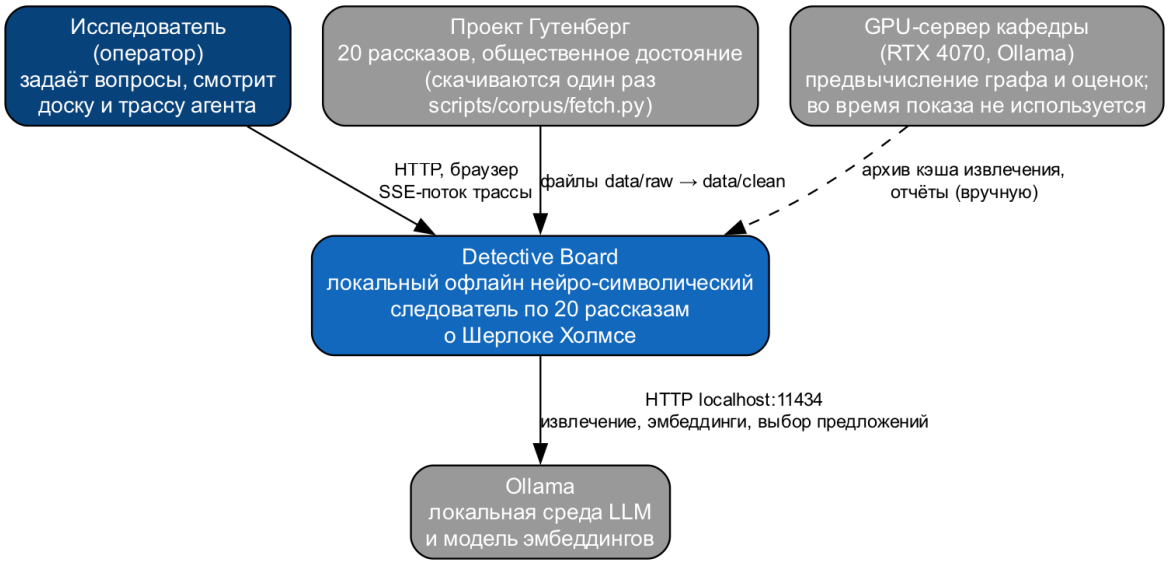



Рисунок 1 – Системный контекст (C4, уровень 1)

Система работает на одной машине. Внешние участники: исследователь (оператор) работает в браузере; Ollama предоставляет локальные модели; корпус загружается с Проекта Гутенберг один раз; GPU-сервер используется только для предварительных вычислений и во время работы системы не нужен.

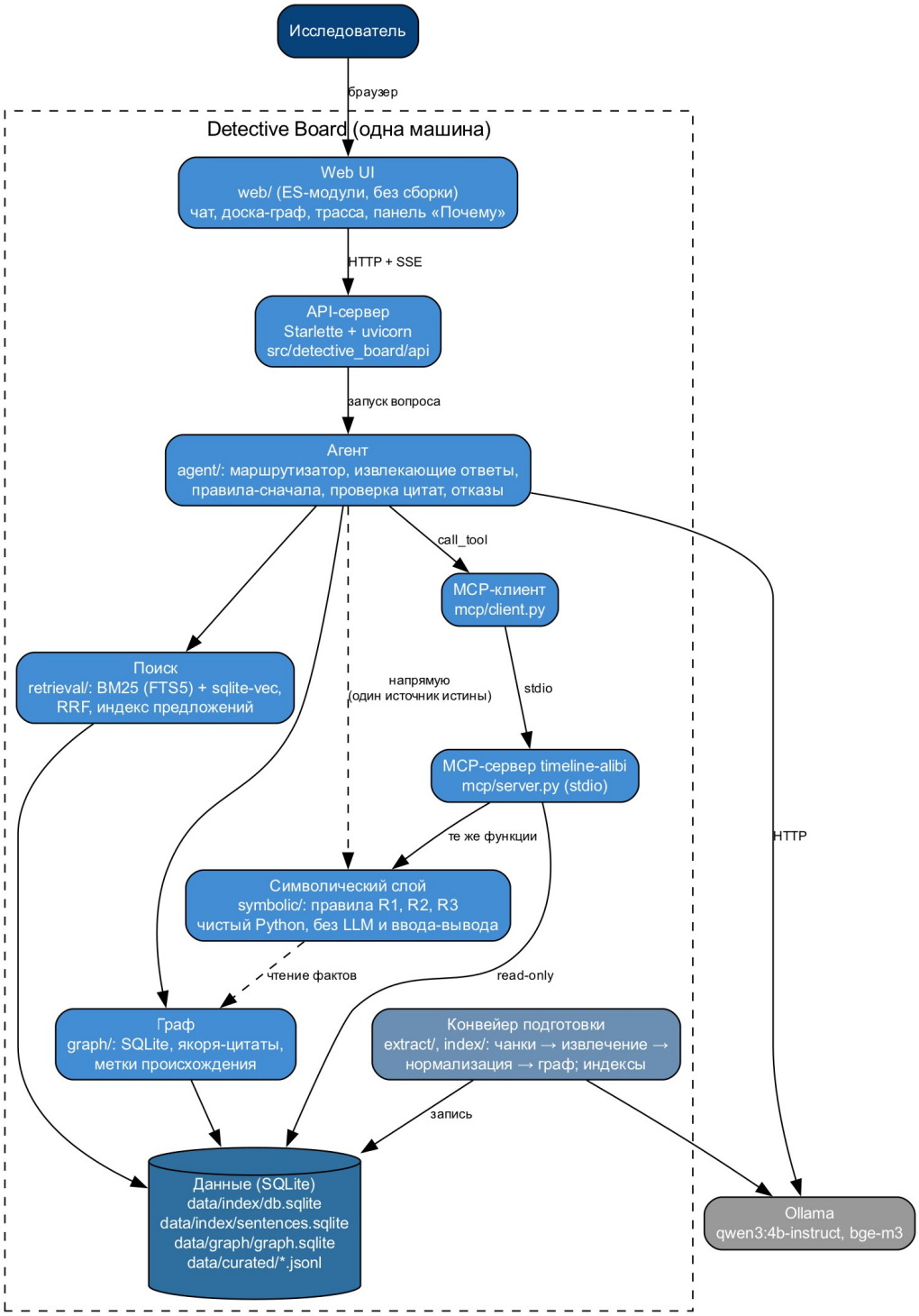



Рисунок 2 – Контейнеры и модули (C4, уровень 2)

Агент и MCP-сервер `timeline-alibi` вызывают одни и те же функции модуля `symbolic/` – результат правила не зависит от пути вызова. MCP-сервер открывает граф только на чтение.

### 3.2 Модули программы

Таблица 2 – Каталоги `src/detective_board/`

| Модуль | Назначение | Ключевые файлы |
|---|---|---|
| `extract/` | Схема извлечения, привязка цитат (`ground.py`), ремонт полей (`repair.py`), нормализация времени и высказываний (`normalise.py`), проход присутствия (`presence*.py`), курируемые факты (`curated.py`), сцены (`scenes.py`), кэш (`cache.py`) | `pipeline.py`, `schema_v2.py`, `run.py` |
| `index/` | Сборка индекса фрагментов и индекса предложений | `build.py`, `sentences.py` |
| `retrieval/` | Эмбеддинги, поиск BM25 + вектор, слияние RRF, индекс предложений | `search.py`, `store.py`, `sentences.py`, `rerank.py` (заглушка) |
| `graph/` | Хранилище графа SQLite, якоря, нормализация, источник фактов для правил | `schema.py`, `store.py`, `facts_source.py`, `timenorm.py` |
| `symbolic/` | Правила R1–R3, модель фактов и результатов, время, места, объяснения | `rules/` `r1_presence.py`, `r2_order.py`, `r3_claims.py`, `api.py` |
| `mcp/` | Клиент и сервер MCP, песочница файлов, ошибки | `server.py`, `client.py`, `tools.py`, `sandbox.py` |
| `agent/` | Маршрутизатор, контекст запроса, извлекающие ответы, ответы «правила сначала», охрана честности, перевод вопроса, память, события | `router.py`, `agent.py`, `extractive.py`, `rule_first.py`, `honesty.py`, `events.py` |
| `api/` | HTTP-сервер: SSE-ответ, граф, источники, состояние; конфигурация; прогрев | `app.py`, `config.py`, `services.py`, `sse.py` |

### 3.3 Поток подготовки данных

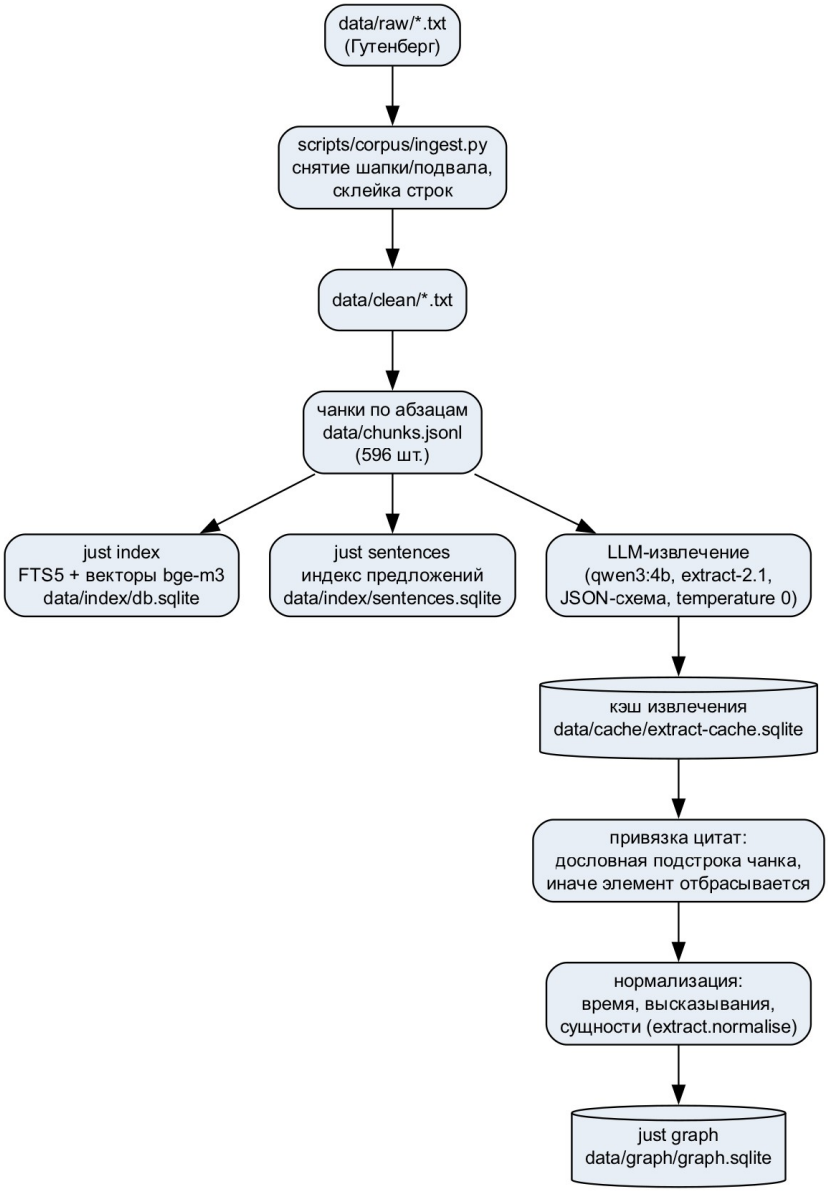



Рисунок 3 – Подготовка данных (выполняется заранее)

Идемпотентность: результат извлечения кэшируется по ключу, в который входят текст фрагмента, тег модели, версия промпта, версия схемы и параметры (`docs/team/models.md`); повторный запуск не вызывает модель. Элемент извлечения, чья цитата не является точной подстрокой фрагмента, отбрасывается и учитывается в таблице `g_dropped`.

### 3.4 Поток обработки вопроса

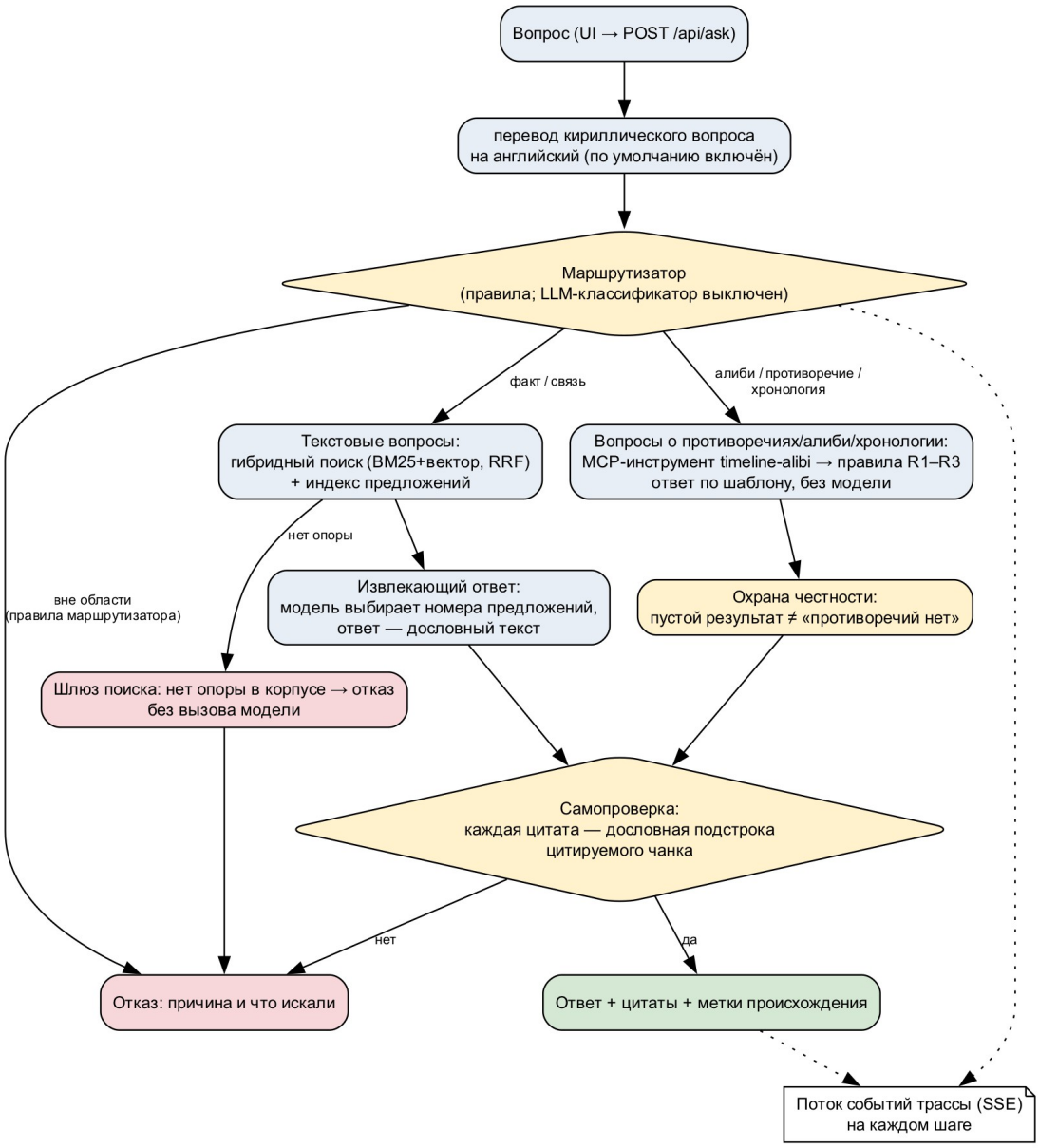



Рисунок 4 – Обработка вопроса агентом

Маршрутизатор – детерминированные регулярные выражения (внекорпусные темы, алиби, противоречия, связи, хронология, таблица присутствия, вопросы о сцене); LLM-классификатор по умолчанию выключен (`use_llm_router = false`) и в любом случае не выбирает инструмент и аргументы. Модель влияет только на формулировку и на выбор номеров предложений.

Таблица 3 – Режимы ответа (`AgentConfig`, `docs/api.md`)

| Режим | Когда | Что делает модель |
|---|---|---|
| Извлекающий (`answer_mode = extractive`, по умолчанию) | Текстовые вопросы: факт, связь, текстовая часть вопросов об алиби/противоречиях | Выбирает 1–2 номера предложений (JSON с ограничением); ответ – их дословный текст, метка `source_text`; при выборе NONE – отказ |
| Правила сначала (`rule_first = true`) | Противоречия, алиби, хронология, таблица присутствия, доска сцены | Не вызывается: ответ – шаблон над результатом инструмента; пустой результат даёт честную фразу о покрытии |
| Генеративный (`answer_mode = generate`) | Только для сравнения (A/B) | Пишет текст ответа с цитатами; самопроверка цитат |
| Отказ | Нет опоры (шлюз: `min_score` 0,018, `min_support` 0,25), внекорпусная тема, не пройдена проверка цитат | Не вызывается при отказе шлюза |

### 3.5 Модель данных

Граф хранится в одном файле SQLite (`data/graph/graph.sqlite`, схема в `src/detective_board/graph/schema.py`, версия построения 3). Узлы: персонаж, место, предмет, событие, высказывание. Каждое ребро и каждый узел события или высказывания несёт якорь – фрагмент, смещения и дословную цитату; сущности привязаны к тексту через таблицу упоминаний.

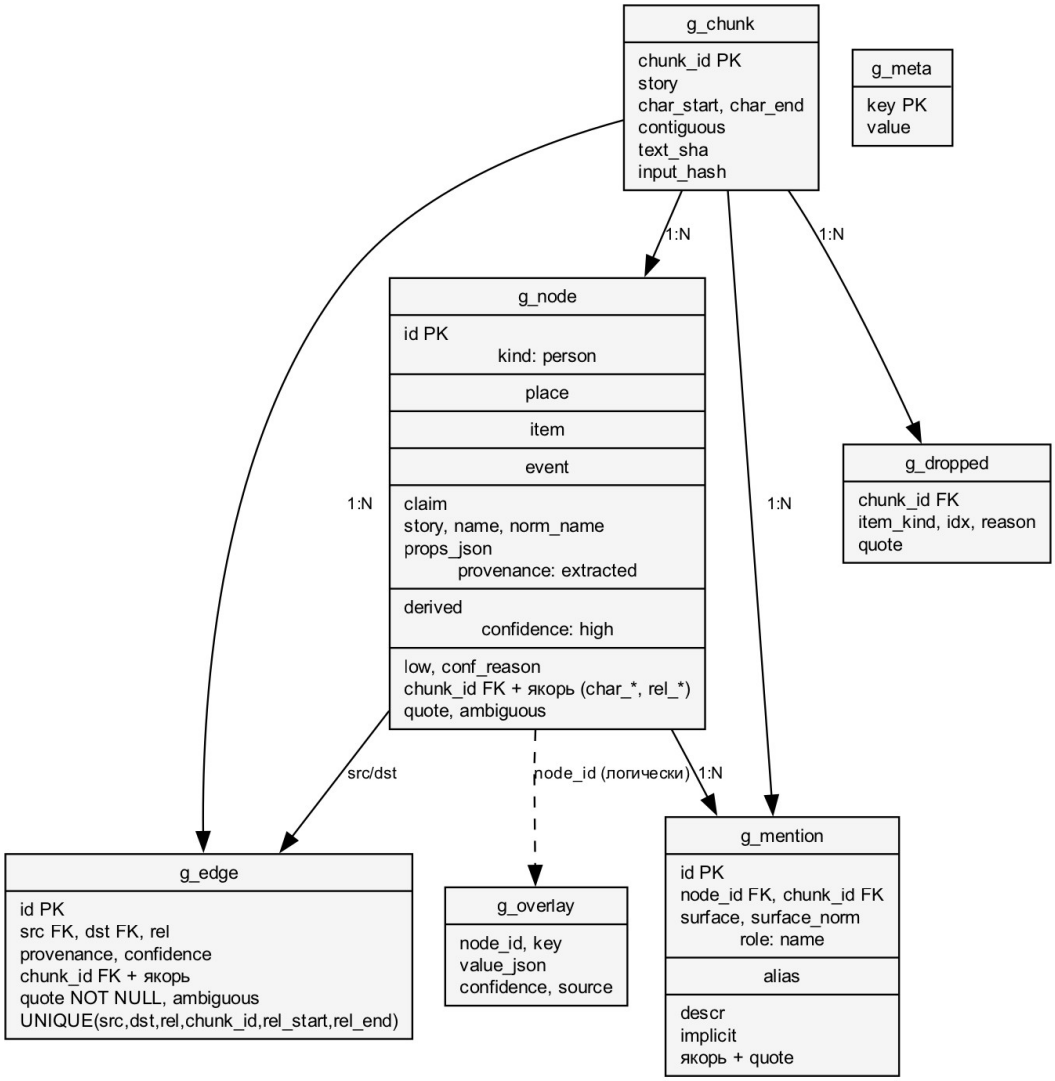



Рисунок 5 – Схема данных графа (SQLite)

Таблица 4 – Таблицы графа

| Таблица | Поля и ограничения |
|---|---|
| `g_chunk` | `chunk_id` PK, `story`, `char_start`, `char_end`, `contiguous`, `text_sha`, `input_hash` (хэш текста, извлечения и версий построения) |
| `g_node` | `id` PK; `kind` ∈ {person, place, item, event, claim}; `story`, `name`, `norm_name`, `unnamed`; `props_json`; `provenance` ∈ {extracted, derived}; `confidence` ∈ {high, low}, `conf_reason`; якорь (`chunk_id`, `char_*`, `rel_*`, `quote`, `ambiguous`) для событий и высказываний |
| `g_edge` | `src`, `dst` → `g_node`; `rel`; `props_json`; `provenance`, `confidence`; якорь и `quote` NOT NULL; уникальность (`src, dst, rel, chunk_id, rel_start, rel_end`) |
| `g_mention` | поверхностные формы сущности во фрагменте: `surface`, `surface_norm`, `role` ∈ {name, alias, descr}, `implicit`, якорь |
| `g_dropped` | аудит отброшенных элементов: `item_kind`, `idx`, `reason`, `quote` |
| `g_overlay` | результаты нормализатора и ручные уточнения, переживающие пересборку: день рассказа, структура высказывания; `confidence`, `source` |
| `g_meta` | служебные пары ключ–значение |

Таблица 5 – Индексы поиска

| Файл | Таблицы |
|---|---|
| `data/index/db.sqlite` | `chunks` (текст и позиции), `chunks_fts` (FTS5, токенайзер porter unicode61), `chunk_vec` (vec0, 1024 измерения, bge-m3), `index_manifest` |
| `data/index/sentences.sqlite` | `sent` (предложения, смещения внутри фрагмента), `sent_fts`, `sent_vec` (vec0 float[1024], косинус), `sent_manifest`; в рассматриваемом графе 11 018 предложений |
| `data/cache/extract-cache.sqlite` | кэш извлечения (ключ → ответ модели) |
| `data/curated/presence_demo.jsonl`, `scenes.jsonl` | курируемые факты присутствия (8 записей) и сцены преступлений (1 сцена) |

Примечание: в `docs/data-schema.md` часть описаний помечена «[P]» (предварительно) и описывает проект; фактической схемой считается код `schema.py`, `retrieval/store.py`, `retrieval/sentences.py`.

### 3.6 Символический слой: правила R1–R3 и метки

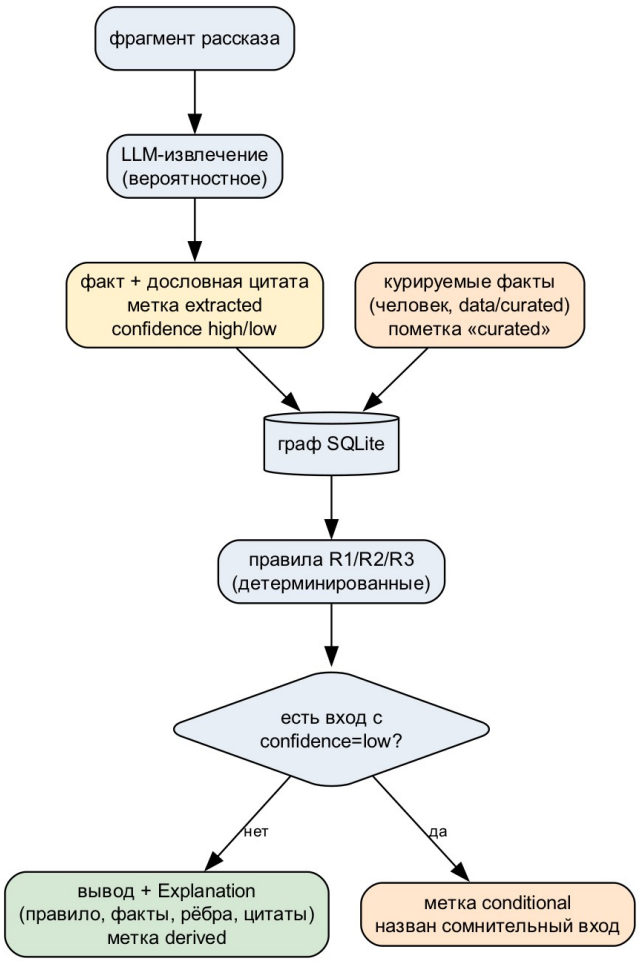



Рисунок 6 – Происхождение утверждений и метки

Таблица 6 – Правила

| Правило | Определение | Файл |
|---|---|---|
| R1 | Для человека p события e1 ≠ e2 с различными местами и определённо пересекающимися интервалами – `violated`; определённо непересекающиеся интервалы или совместимые места – `consistent`; иначе `unknown` с причиной (`PLACE_UNRESOLVED`, `TIME_UNDECIDED`, …). Режимы: `scan_presence_conflicts`, `check_presence` (алиби) | `symbolic/rules/r1_presence.py` |
| R2 | Граф «раньше» из относительных меток (`after`/`before`) и абсолютных времён; транзитивное замыкание; цикл (компонента сильной связности) – `violated`; пара без пути – `unknown` (`NO_PATH`). Функции: `find_order_cycles`, `order_of`, `timeline` | `symbolic/rules/r2_order.py` |
| R3 | Структурированное утверждение (о присутствии или порядке) против установленного факта: событие в голосе рассказчика, уверенность high. Несовместимость – `violated` (`claim_incompatible_with_fact`), без обвинения свидетеля во лжи; неструктурированное утверждение – `unknown` | `symbolic/rules/r3_claims.py` |

Перекрытие интервалов: для непрерывных – полуоткрытые (касание не перекрытие), если один из интервалов – момент, то замкнутые. Определённое перекрытие требует точного времени и известного дня; относительное время («на следующее утро») может доказать непересечение разных дней, но не определённое перекрытие. Время в пути между местами не моделируется. Совместимость мест определяется таблицей алиасов и отношением «содержит», а не сходством строк (`docs/symbolic-layer.md`, разделы 4–5).

Объект объяснения (`Explanation`) содержит: идентификатор и версию правила, вердикт, признак `conditional`, код причины, субъект, использованные факты и рёбра (каждый с якорем и цитатой), шаги вывода, множество зависимых извлечённых фактов, слабые факты (`weak`), недостающее (`missing`) и примечания. Нарушение без якоря – ошибка (инвариант документа `docs/symbolic-layer.md`, раздел 3.1).

### 3.7 MCP-контракт

Собственный сервер `timeline-alibi` (stdio, `src/detective_board/mcp/server.py`) строится на SDK `mcp` и содержит

шесть инструментов; контракт – `docs/mcp-contracts.md`. Агент вызывает их через `mcp/client.py` (белый список, тайм-ауты, отображение ошибок).

Таблица 7 – Инструменты сервера timeline-alibi

| Инструмент | Вход (основное) | Выход (основное) |
|---|---|---|
| `build_timeline` | story, person_id?, place_id?, include_undated, limit ≤ 200 | status ok / insufficient_evidence; события с интервалом, местом, участниками, якорями; `undated_count, total_events, curated` |
| `check_alibi` | story, person_id, window, claimed_place_id? | status supported / contradicted / unknown / insufficient_evidence; свидетельства; provenance derived / conditional; explanation (R1) |
| `find_overlaps` | story, person_ids ≤ 10, same_place_only, window?, limit ≤ 100 | пары пересекающихся событий с якорями, provenance, explanation (R1) |
| `find_contradictions` | story, person_id?, rules? (R1/R2/R3), limit | findings (правило, provenance, explanation, якоря); пустой список с ok ≠ «согласовано» |
| `presence_table` | story, person_id, names?, limit ≤ 100 | строки «где, по чьим словам, цитата», утверждение или отрицание; конфликты R1 |
| `scene_board` | story, place, window, person_ids?, victim_ids?, limit ≤ 50 | статусы persons: excluded / placed / not_excluded; счётчики; без суждения о виновности |

Модель ошибок: домены возвращают обычный результат со статусом; сбои протокола и проверки – объект `{error: {code, message, retriable, details}}`, коды `invalid_input`, `not_found`, `ambiguous`, `insufficient_evidence`, `timeout`, `store_unavailable`, `internal` (`docs/mcp-contracts.md`, раздел 5). Серверный предел времени вызова – 4 с (`DeadlineRunner`; `DETECTIVE_BOARD_TOOL_DEADLINE_S`); значения лимитов в контракте помечены как предварительные. Пути чтения файлов ограничены песочницей `mcp/sandbox.py`.

Планировавшийся сервер заметок `evidence-notes` и готовые серверы filesystem/fetch в реализации отсутствуют.

### 3.8 HTTP-интерфейс

Таблица 8 – Маршруты (src/detective_board/api/app.py, docs/api.md)

| Маршрут | Назначение |
|---|---|
| POST /api/ask | Запуск вопроса; ответ text/event-stream, по одному событию на шаг; тело JSON ≤ 16 КиБ, вопрос 1–1000 символов, история ≤ 6 пар; второй одновременный запуск – 429 busy |
| GET /api/graph/subgraph | Окрестность сущности (0–3 шага, ≤ 200 узлов) с якорями и объяснениями производных рёбер |
| GET /api/graph/entities | Поиск сущностей для подсказок |
| GET /api/source/{chunk_id} | Текст фрагмента с подсветкой цитаты (идентификатор проверяется по шаблону, путь к файлу не используется) |
| GET /api/health | Состояние: Ollama, модели, индекс, граф, MCP, прогрев |
| GET /api/config | Эффективные несекретные настройки и перечень событий |
| / (статика) | Интерфейс из web/ |

События SSE (agent/events.py): query_translated, route_decided, retrieval_started, retrieval_results, graph_query, graph_results, tool_call, tool_result, enriched_query, answer_token, citation_check, refusal, final; при сбое вместо final приходит run_error с кодом internal, unavailable или timeout. Итоговое событие содержит статус (answered, partial, refused), признак verified, ответ, цитаты с якорями и метки происхождения.

### 3.9 Веб-интерфейс

Статические файлы web/ (ES-модули app.js, graph.js, trace.js, replay.js, demo.js и др., библиотеки в web/vendor/): строка вопроса и подсказки, панель трассы шагов, доска-граф с карточками сущностей и нитями, панель ответа с цитатами и меткой происхождения, панель «Почему» с объяснением правила, режимы «Записи» и демонстрационные подсказки. Записи прогонов (web/replay/*.json) в интерфейсе помечены как записи, а не живой прогон. Горячие клавиши: / – фокус на вопрос, g – список вместо графа, c – сохранённый прогон, t – последнее событие трассы, Esc – сброс. Подробно – в «Руководстве оператора» (РО).

## 4 ИСПОЛЬЗУЕМЫЕ ТЕХНИЧЕСКИЕ СРЕДСТВА

Целевое: ноутбук ThinkPad X1 Carbon 7 поколения (i7-8665U, 4 ядра, 15 ГиБ ОЗУ, без GPU), Linux x86_64. Для предварительных вычислений использовался сервер кафедры (i7-12700K, 96 ГБ, RTX 4070 12 ГБ). Развёртывание показано на рисунке «Схема развёртывания».

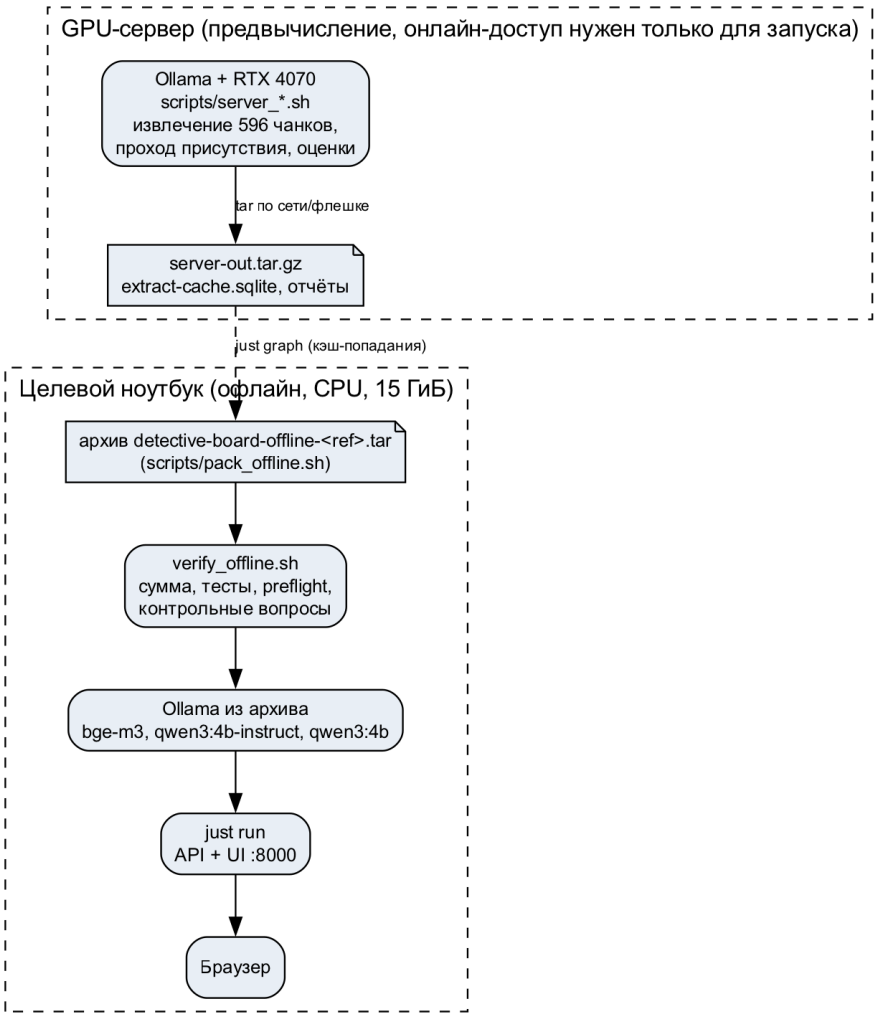



Рисунок 7 – Схема развёртывания: GPU-сервер и целевой ноутбук

## 5 ВЫЗОВ И ЗАГРУЗКА

Запуск: `just run` (предварительно проверяет доступность Ollama и моделей скриптом `scripts/preflight.py`, затем выполняет `python -m detective_board.api`, адрес http://127.0.0.1:8000). `just run --fake` запускает интерфейс с фиктивной моделью и тремя заготовленными фрагментами без Ollama и без файлов данных. Загрузка индекса, графа и MCP-сервера происходит при старте; прогрев моделей – `api/warmup.py`.

## 6 ВХОДНЫЕ ДАННЫЕ

– Вопрос пользователя (до 1000 символов) и история до 6 пар «вопрос –ответ» (`POST /api/ask`).
– Корпус: `data/clean/*.txt`, `data/chunks.jsonl`; индексы `data/index/db.sqlite`, `data/index/sentences.sqlite`; граф `data/graph/graph.sqlite`; курируемые файлы `data/curated/*.jsonl`.
– Конфигурация: значения по умолчанию < TOML-файл (`--config` или `DETECTIVE_BOARD_CONFIG`) < переменные `DETECTIVE_BOARD_<ПОЛЕ>` < параметры командной строки.

## 7 ВЫХОДНЫЕ ДАННЫЕ

– Поток событий SSE и итоговое событие `final` (ответ, цитаты с якорями, метки происхождения, статус).
– Объекты `Explanation` для выводов правил; подграф и текст фрагментов для интерфейса.
– Отчёты оценки: `docs/spike/results/eval-<дата>-<срез>-<sha7>-<хэш конфигурации>-<время>.{md,json,svg}` (ни один результат не перезаписывается).

## ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

| **Изм.** | **Номера листов (страниц)** | | | **Всего листов** | **№ докум.** | **Подп.** | **Дата** |
|---|---|---|---|---|---|---|---|
| | изменённых | замененных | новых | | | | |
| | | | | | | | |
| | | | | | | | |
| | | | | | | | |
| | | | | | | | |